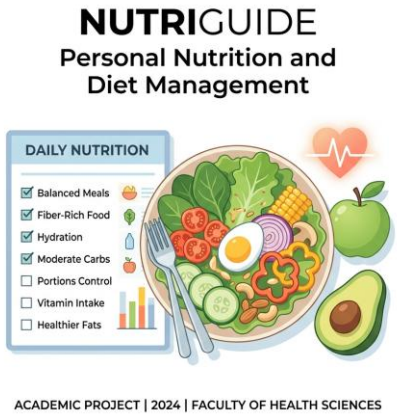
॥ वसुधैव कुटुम्बकम् ॥

A DBMS MINI PROJECT REPORT ON

NutriGuide — Your Personal Nutrition Companion



Batch: B3 | Group: GRP-1

Submitted by:

Tejas Jagdish Patil (26070122510)

Veer Parmar (26070122508)

Postiwala Vatsal Dipakbhai (26070122509)

Parth Bhayani (26070122512)

Under the Guidance of

Dr. Balkrishna Patil

Department of Computer Science

Symbiosis Institute of Technology (SIT), Pune

Symbiosis International (Deemed University)

Academic Year 2026–2027

INDEX

Sr. No.	Title	Page No.
1.	**Introduction**	2
2.	**Problem Statement (Step 1)**	3
3.	**System Architecture and Modules**	4
4.	**Functional and Non-Functional Requirements (Step 2)**	5–6
5.	**Entities, Relationships and Attributes (Step 3)**	7–8
6.	**ER and EER Diagrams (Step 4)**	9–10
7.	**Relational Schema (Step 5)**	11
8.	**Anomalies (Step 6)**	12
9.	**Functional Dependencies (Step 7)**	13–14
10.	**Normalization (Step 8: 1NF to BCNF)**	15–17
11.	**Implementation of Tables in MySQL (Step 9)**	18–22
12.	**Functions, Procedures, Triggers and Queries (Step 10)**	23–30
13.	**Application of Codd's 12 Rules**	31–32
14.	**Individual Contributions (Step 11)**	33
15.	**Conclusion**	34
16.	**References**	35
App. A	**Complete SQL Script (`NutriGuide.sql`)**	35–36
App. B	**Stored Functions, Procedures, Views & Triggers**	37–38

1. Introduction

In modern preventive healthcare and personal wellness management, dietary intake and nutrient balance play a pivotal role. The widespread availability of processed foods, irregular eating routines, and diverging nutritional needs across individuals have created an urgent imperative for structured dietary tracking systems. Many individuals attempt to monitor their caloric and macronutrient intake (proteins, carbohydrates, fats) using manual logbooks, generic smartphone note apps, or disconnected spreadsheets. However, manual recording systems quickly deteriorate: mathematical aggregation is labor-intensive, multi-item meals are recorded with varying serving definitions, and longitudinal history becomes fragmented or lost.

NutriGuide — Your Personal Nutrition Companion is a comprehensive, production-grade relational database management system designed to systematically capture, organize, analyze, and optimize personal nutrition data.

Engineered specifically for academic evaluation and clinical-demonstration rigor under the Database Management Systems (DBMS) curriculum, NutriGuide provides an enterprise-quality relational backend running on the **MySQL 8.x** relational database engine with InnoDB ACID transactional semantics.

The database models ten tightly integrated entities spanning:

1. **User Demographics & Anthropometrics**: Secure user credentials, demographic parameters (age, gender), physical measurements (height in centimeters, weight in kilograms), and physical activity levels.
2. **Macronutrient & Caloric Target Goals**: User-defined nutritional targets (calories, protein, carbohydrates, fat) with historical versioning and single-active-goal business rule enforcement.
3. **Reference Food Catalog**: Categorized food items (Grains, Proteins, Dairy, Fruits, Vegetables) with precise nutritional densities per standard serving size.
4. **Multi-Item Meal Transactions**: Real-time logging of breakfast, lunch, dinner, and snack meal events, resolved atomically through an associative junction entity (`meal_item`).
5. **Prospective Meal Planning**: Weekly and monthly diet regimens scheduled across date intervals, linked to designated target meal slots.
6. **Rule-Based Recommendation History**: Persisted dietary advisory guidelines evaluating user caloric deficits, surpluses, and protein compliance against standard WHO nutritional benchmarks.
7. **Automated Audit Logging**: Real-time audit trails maintained via database triggers capturing user operations without application-side overhead.

The core relational database is implemented in MySQL 8.x, enforcing schema stability through primary keys, foreign keys with explicit referential integrity actions (`ON UPDATE CASCADE`, `ON DELETE CASCADE`, `ON DELETE RESTRICT`), candidate key uniqueness, and declarative CHECK constraints. Above the table storage layer, NutriGuide incorporates high-level database programmability including stored functions for dynamic Body Mass Index (BMI) computation, stored procedures with output parameters for aggregate daily nutrition summaries, analytical views, and audit/validation triggers.

2. Problem Statement (Step 1)

Individuals and dietary consultants seeking to manage nutritional health through manual record-keeping face several systematic data management challenges:

8. **Scattered and Disorganized Records**: Dietary intake, body weight fluctuations, target goals, and food nutrient values are frequently noted in disparate physical notebooks, phone memos, or isolated spreadsheets. As a result, compiling a consolidated, longitudinal picture of an individual's nutritional health is virtually impossible.
9. **Nutrient Calculation Errors & Latency**: Computing cumulative daily calories and macro splits (grams of protein, carbohydrates, and fats) requires repetitive arithmetic scaling based on portion size. Manual calculations are prone to severe calculation errors, misleading users regarding their actual intake.
10. **Data Redundancy & Inconsistency (Flat File Flaws)**: When nutritional data is stored in unnormalized flat files, food descriptions and nutrient densities are repeatedly duplicated across every logged meal. If the nutritional profile of an item (e.g., Cooked White Rice) is updated, records in earlier rows remain unchanged, creating contradictory records.
11. **Lack of Validation & Boundary Integrity**: Manual systems fail to prevent erroneous inputs such as negative serving sizes, negative weights or heights, or impossible calorie amounts.
12. **Absence of Transactional Atomicity**: A meal typically consists of multiple foods consumed together. In traditional spreadsheets, a partial crash during entry leaves orphaned records where a meal is recorded without its accompanying food components.
13. **No Automated Audit History**: Manual systems provide no reliable mechanism to detect when a meal was logged, modified, or deleted, impeding auditability and compliance tracking.
14. **Disconnected Goal Analysis**: Without automated relational joins between logged intake and active goals, users cannot deterministically evaluate whether they are in a caloric surplus or deficit.

Proposed Solution

NutriGuide resolves each of these challenges through a centralized, strictly normalized relational database. The schema eliminates data redundancy, guarantees referential and transactional integrity, automates dynamic metric calculations through stored subprograms, enforces data validation at the storage layer via CHECK constraints and triggers, and enables instantaneous reporting through pre-aggregated SQL views.

3. System Architecture and Modules

3.1 Three-Tier Relational Topology

The NutriGuide architecture is structured into three clean, decoupled tiers:

15. **Presentation Layer (Java Swing Client)**: An ergonomic Windows desktop user interface built using Java Swing. It provides dedicated user interface windows (`LoginFrame`, `RegisterFrame`, `DashboardFrame`, `MealLogFrame`, `NutritionGoalFrame`, `RecommendationFrame`, `ReportFrame`, `ProfileFrame`) running on the Swing Event Dispatch Thread (EDT). All long-running API operations are dispatched asynchronously using `SwingWorker` threads to prevent user interface freezing.
16. **Application & Logic Layer (Java HttpServer)**: A lightweight, high-performance HTTP REST server running on `http://127.0.0.1:8081`. It encapsulates the application services (`AuthenticationService`, `NutritionService`, `MealService`, `FoodService`, `RecommendationService`, `ReportService`, `UserService`), token-based `SessionManager`, and salted `PBKDF2-HMAC-SHA256` password security. It issues parameterized JDBC queries to MySQL.
17. **Relational Database Layer (MySQL 8.x Engine)**: The relational database tier manages table storage using the InnoDB storage engine. It provides ACID transactional guarantees, enforces relational integrity, executes stored functions and procedures, maintains aggregation views, and automatically invokes audit and validation triggers.

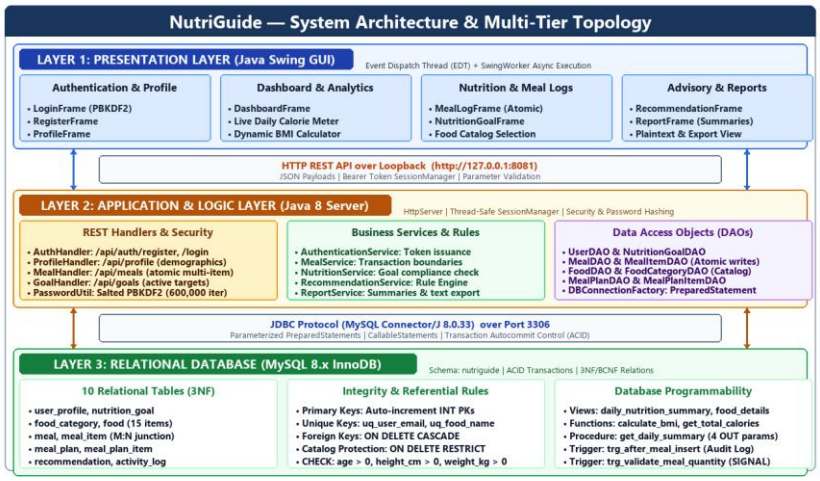


Figure 3.1: NutriGuide System Architecture & Multi-Tier Topology

3.2 System Modules

18. **User Profile & Authentication Module (`user_profile`)**:

- Manages user account intake, authentication, and secure demographic tracking.
- Stores user height, weight, age, gender, and physical activity tier.
- Protects user credentials using salted PBKDF2-HMAC-SHA256 password digests with an adaptive 600,000-iteration work factor.

19. **Nutrition Goal Management Module (`nutrition_goal`)**:

- Allows users to configure custom daily caloric and macronutrient targets.
- Implements historical versioning through an `is_active` status flag.

- Enforces the business constraint that exactly one goal may remain active per user at any given timestamp.

20. **Food Catalog & Categorization Module (`food_category`, `food`)**:

- Maintains a standardized, immutable reference catalog of foods classified into primary dietary groups (Grains, Proteins, Dairy, Fruits, Vegetables).
- Tracks standard serving sizes in grams and associated energy/macronutrient values (calories, protein, carbs, fat).

21. **Meal Logging & Transaction Module (`meal`, `meal_item`)**:

- Captures daily meal events (Breakfast, Lunch, Dinner, Snack).
- Utilizes an associative junction entity (`meal_item`) to resolve the many-to-many relationship between meals and food items.
- Enforces ACID transactions with atomic commit and rollback capabilities.

22. **Meal Planning Module (`meal_plan`, `meal_plan_item`)**:

- Enables prospective diet regimen scheduling across structured date windows (e.g., 7-day weight cut plan, 30-day muscle gain plan).
- Resolves planned meal schedules against food catalog entries.

23. **Rule-Based Recommendation Engine (`recommendation`)**:

- Evaluates active nutritional goals against actual aggregate daily intake.
- Deterministically generates clinically grounded educational guidance regarding caloric deficits, surpluses, and protein adequacy.

24. **Automated Audit & Logging Module (`activity_log`)**:

- Maintains a tamper-evident audit history of user lifecycle actions and trigger-logged meal transactions.

25. **Reporting & Analytics Module (`views`, `procedures`)**:

- Leverages SQL views (`daily_nutrition_summary`, `food_category_details`) and stored procedures (`get_daily_summary`) to generate instantaneous analytical reports for date ranges.

4. Functional and Non-Functional Requirements (Step 2)

4.1 Functional Requirements

Requirement ID	Functional Description	Implemented Database Object
FR1	Store and retrieve user profiles including height, weight, age, gender, and activity level.	Table `user_profile`
FR2	Securely hash user passwords using salted PBKDF2 with 600,000 iterations; prevent plaintext leakage.	Table `user_profile.password_hash`, `PasswordUtil.java`
FR3	Enforce global uniqueness of user email addresses to prevent duplicate registrations.	Constraint `UNIQUE KEY uq_user_email` on `user_profile`
FR4	Maintain food categories to systematically classify nutritional catalog items.	Table `food_category`
FR5	Store verified food items with per-serving caloric, protein, carbohydrate, and fat densities.	Table `food` (FK `category_id`)

Requirement ID	Functional Description	Implemented Database Object
FR6	Enforce uniqueness of food names to avoid duplicate food item creation.	Constraint `UNIQUE KEY uq_food_name` on `food`
FR7	Prevent physical deletion of food items referenced in past meal records to preserve historical integrity.	Referential Action `ON DELETE RESTRICT` on `meal_item.food_id`
FR8	Allow users to establish macronutrient and caloric target goals with historical archiving.	Table `nutrition_goal`
FR9	Enforce that only one nutrition goal is active (`is_active = TRUE`) per user account.	Service Layer Transaction & Table `nutrition_goal`
FR10	Record meal events linked to an authenticated user with meal type and date.	Table `meal` (FK `user_id`)
FR11	Atomically record multi-item meal lines linked to a parent meal and food catalog item.	Table `meal_item` (Composite/Surrogate PK, FKs)
FR12	Automatically log meal insertion events into the audit log without application intervention.	Trigger `trg_after_meal_insert` on `meal`
FR13	Reject invalid or non-positive food serving quantities at the database engine level.	Trigger `trg_validate_meal_quantity` (SIGNAL '45000')
FR14	Automatically compute user Body Mass Index (BMI) dynamically via a stored database function.	Stored Function `calculate_bmi(weight_kg, height_cm)`
FR15	Provide instantaneous multi-table aggregated daily nutritional intake totals.	Relational View `daily_nutrition_summary`
FR16	Return consolidated daily macro totals via stored procedure with output parameters.	Stored Procedure `get_daily_summary(...)`
FR17	Store prospective multi-day diet plans and their component planned meal items.	Tables `meal_plan`, `meal_plan_item`
FR18	Automatically clean up all dependent records (goals, meals, plans, logs) upon user profile deletion.	Referential Action `ON DELETE CASCADE` across foreign keys

4.2 Non-Functional Requirements

- **Data Integrity & Consistency**: Enforce strict schema constraints including declarative PRIMARY KEYs, FOREIGN KEYs with explicit referential integrity rules, and CHECK constraints guarding against negative physical metrics.

- **ACID Transaction Compliance**: Multi-item meal logging and goal transitions must execute within explicit atomic transaction boundaries with rollback on exception.
- **Precision of Nutritional Data**: All caloric and macronutrient quantities must use high-precision fixed-point numeric types (`DECIMAL(7,2)`, `DECIMAL(6,2)`, `DECIMAL(5,2)`) instead of floating-point representations (`FLOAT`, `DOUBLE`) to prevent IEEE 754 rounding drift.
- **Architectural Security**: The database binds strictly to local loops; passwords are never persisted or returned in plaintext; all SQL queries utilize parameterized statements to prevent SQL injection.
- **Modularity & High Cohesion**: Schema separation strictly adheres to Third Normal Form (3NF), ensuring that tables correspond to discrete real-world business entities.

5. Entities, Relationships and Attributes (Step 3)

5.1 Entities and Attributes

Entity / Table Name	Primary Key	Foreign Keys	Key Attributes	Business Purpose
`user_profile`	`user_id`	*None*	`name`, `email`, `password_hash`, `age`, `gender`, `height_cm`, `weight_kg`, `activity_level`, `created_at`	Stores registered user demographic, anthropometric, and credential records.
`nutrition_goal`	`goal_id`	`user_id`	`target_calories`, `target_protein`, `target_carbs`, `target_fats`, `is_active`, `created_at`	Stores caloric and macronutrient daily targets configured by users.
`food_category`	`category_id`	*None*	`category_name`	Classifies food items into distinct dietary classifications.
`food`	`food_id`	`category_id`	`food_name`, `serving_size`, `calories`, `protein`, `carbohydrates`, `fat`	Reference catalog of consumable foods and their nutrient densities.
`meal`	`meal_id`	`user_id`	`meal_type`, `meal_date`, `created_at`	Represents a discrete meal event consumed by a user on a specific date.
`meal_item`	`meal_item_id`	`meal_id`, `food_id`	`quantity`	Associative junction resolving the M:N relationship between meals and foods.
`meal_plan`	`meal_plan_id`	`user_id`	`plan_name`, `start_date`, `end_date`, `is_active`, `created_at`	Prospective diet regimen schedules established over date intervals.

Entity / Table Name	Primary Key	Foreign Keys	Key Attributes	Business Purpose
`meal_plan_item`	meal_plan_item_id	meal_plan_id, food_id	planned_meal_type, quantity	Associative junction resolving the M:N relationship between diet plans and foods.
`recommendation`	rec_id	user_id	rec_type, message, generated_at	Persisted rule-based educational guidelines and advisories.
`activity_log`	log_id	user_id	action_taken, log_time	Audit trail capturing user system interactions and trigger-logged meal transactions.

5.2 Relationships and Cardinalities

Relationship	Entity 1	Entity 2	Cardinality	Participation	Description
User_Sets_Goals	user_profile	nutrition_goal	**1 : N**	User: Partial Goal: Total	One user can configure multiple historical goals; each goal belongs to exactly one user.
User_Logs_Meals	user_profile	meal	**1 : N**	User: Partial Meal: Total	A user can log zero, one, or many meals; each meal is logged by exactly one user.
Meal_Contains_Items	meal	meal_item	**1 : N**	Meal: Total Item: Total	A meal contains one or more meal items; each line item belongs to one meal header.
Food_Appears_In_Items	food	meal_item	**1 : N**	Food: Partial Item: Total	A food can appear in multiple meal items; each line item references one specific food.
Category_Classifies_Food	food_category	food	**1 : N**	Category: Partial Food: Total	A category groups multiple foods; each food belongs to exactly one category.
User_Schedules_Plans	user_profile	meal_plan	**1 : N**	User: Partial Plan: Total	A user can create zero, one, or many prospective diet plans.

Relationship	Entity 1	Entity 2	Cardinality	Participation	Description
Plan_Includes_Items	meal_plan	meal_plan_item	**1 : N**	Plan: Total Item: Total	A meal plan incorporates one or more scheduled food entries.
Food_In_Meal_Plans	food	meal_plan_item	**1 : N**	Food: Partial Item: Total	A food can be scheduled across multiple diet plans.
User_Receives_Recs	user_profile	recommendation	**1 : N**	User: Partial Rec: Total	A user can receive multiple tailored recommendations over time.
User_Generates_Audit	user_profile	activity_log	**1 : N**	User: Partial Log: Total	User events and trigger executions generate audit records in the activity log.

Resolution of Many-to-Many Relationships

- **Meal and Food (M:N)**: A meal can consist of multiple foods (e.g., Rice, Dal, Salad), and a single food can be consumed across thousands of different meals. This M:N cardinality is decomposed into two 1:N relationships via the **`meal_item`** associative entity with `quantity` as an intersection attribute.
- **Meal_Plan and Food (M:N)**: A prospective diet plan contains multiple planned foods, and a food can be scheduled across numerous diet plans. This M:N relationship is resolved into two 1:N relationships via the **`meal_plan_item`** associative entity.

6. ER and EER Diagrams (Step 4)

6.1 Conceptual ER Diagram (Chen Notation)

The Entity-Relationship (ER) diagram below illustrates the conceptual schema of NutriGuide following standard Chen notation:

- **Rectangles**: Represent the ten fundamental entity sets.
- **Diamonds**: Represent relationship sets linking entities.
- **Ellipses**: Represent entity attributes, with primary keys underlined and unique constraints indicated.
- **Numeric Cardinalities**: Expressed along the relationship lines (`1` and `N`).

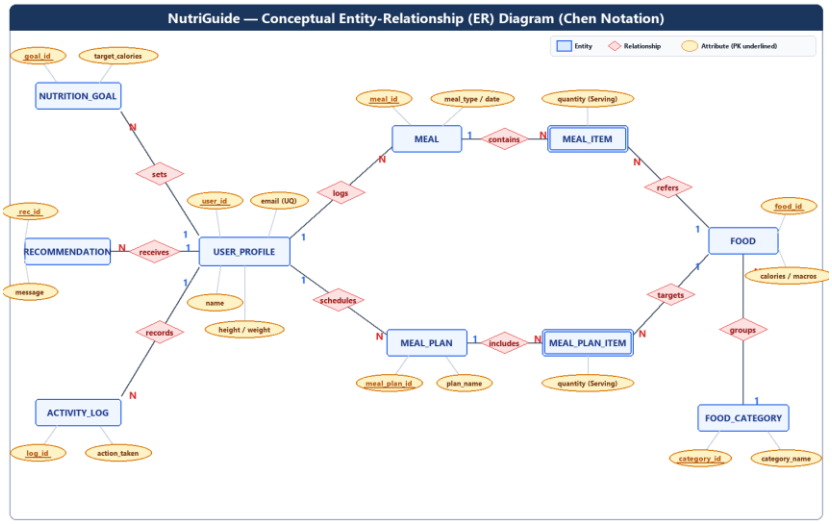


Figure 6.1: NutriGuide Conceptual ER Diagram (Chen Notation) with M:N Associative Resolution

6.2 Enhanced Entity-Relationship (EER) Diagram

The Enhanced ER (EER) diagram provides full physical implementation specifics:

- Detailed schema table cards with explicit data types, primary keys, and foreign key linkages.
- Exact referential integrity constraints (`ON DELETE CASCADE`, `ON DELETE RESTRICT`, `ON UPDATE CASCADE`).
- Declarative CHECK constraints governing demographic and nutritional boundaries.
- The trigger-maintained audit relationship: a dashed pipeline illustrating `trg_after_meal_insert` automatically populating `activity_log`.

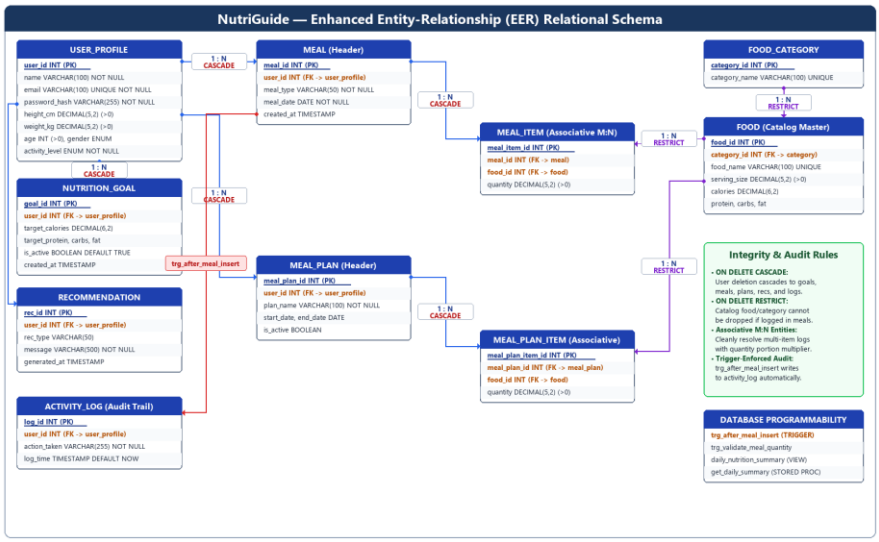


Figure 6.2: NutriGuide Enhanced Entity-Relationship (EER) Relational Schema with Constraints and Foreign Keys

7. Relational Schema (Step 5)

7.1 Relational Notation

(Underlined: Primary Key | Italic: Foreign Key)

26. **`user_profile`** (user_id, name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at)
27. **`nutrition_goal`** (goal_id, *user_id*, target_calories, target_protein, target_carbs, target_fats, is_active, created_at)
28. **`food_category`** (category_id, category_name)

29. **`food`** (food_id, *category_id*, food_name, serving_size, calories, protein, carbohydrates, fat)
30. **`meal`** (meal_id, *user_id*, meal_type, meal_date, created_at)
31. **`meal_item`** (meal_item_id, *meal_id*, *food_id*, quantity)
32. **`meal_plan`** (meal_plan_id, *user_id*, plan_name, start_date, end_date, is_active, created_at)
33. **`meal_plan_item`** (meal_plan_item_id, *meal_plan_id*, *food_id*, planned_meal_type, quantity)
34. **`recommendation`** (rec_id, *user_id*, rec_type, message, generated_at)
35. **`activity_log`** (log_id, *user_id*, action_taken, log_time)

7.2 Foreign Keys and Referential Actions

Foreign Key	Child Table	Parent Table & Referenced Column	ON UPDATE	ON DELETE	Rationale
fk_nutrition_goal_user	nutrition_goal	user_profile(user_id)	CASCADE	CASCADE	Deleting a user profile removes their historical goal records.
fk_food_category	food	food_category(category_id)	CASCADE	RESTRICT	Prevents accidental deletion of categories that currently contain active foods.
fk_meal_user	meal	user_profile(user_id)	CASCADE	CASCADE	Deleting a user profile cleans up all associated meal headers.
fk_meal_item_meal	meal_item	meal(meal_id)	CASCADE	CASCADE	Deleting a meal event automatically cascades to remove its constituent line items.
fk_meal_item_food	meal_item	food(food_id)	CASCADE	RESTRICT	Prevents deleting a catalog food item if it has been logged in user history.
fk_meal_plan_user	meal_plan	user_profile(user_id)	CASCADE	CASCADE	Removing a user account removes all scheduled diet plans.
fk_meal_plan_item_plan	meal_plan_item	meal_plan(meal_plan_id)	CASCADE	CASCADE	Removing a diet plan removes all associated planned item entries.

Foreign Key	Child Table	Parent Table & Referenced Column	ON UPDATE	ON DELETE	Rationale
fk_meal_plan_item_food	meal_plan_item	food(food_id)	CASCADE	RESTRICT	Prevents deleting foods that are actively scheduled in future diet regimens.
fk_recommendation_user	recommendation	user_profile(user_id)	CASCADE	CASCADE	Purging a user cleans up all historical recommendations.
fk_activity_log_user	activity_log	user_profile(user_id)	CASCADE	CASCADE	Account termination cascades to clean up the user's audit log entries.

8. Anomalies (Step 6)

A database anomaly is a data integrity flaw that manifests when unrelated business facts are merged into a single unnormalized relation. To illustrate why NutriGuide must be decomposed into multiple normalized relations, consider an unnormalized "flat" design (NUTRITION_FLAT) where user profiles, meal events, food catalog attributes, and food categories are merged into a single table:

Unnormalized Relation: `NUTRITION_FLAT`

(Composite Candidate Key: `{email, meal_date, meal_type, food_name}`)

user_id	name	email	height_cm	weight_kg	meal_date	meal_type	food_name	serving_size	calories	protein	category_name	quantity
1	John Doe	user@example.com	175.00	70.00	2026-10-18	Lunch	Grilled Chicken	100.00	165.00	31.00	Proteins	1.00
1	John Doe	user@example.com	175.00	70.00	2026-10-18	Lunch	Cooked White Rice	100.00	130.00	2.70	Grains	1.50
1	John Doe	user@example.com	175.00	70.00	2026-10-18	Dinner	Yellow Dal	100.00	116.00	7.30	Proteins	1.00
2	Jane Smith	jane@example.com	162.00	58.00	2026-10-18	Breakfast	Rolled Oats	100.00	389.00	16.90	Grains	1.00

8.1 Insertion Anomalies

36. **Inability to Add Foods Without Meals**: If a dietitian adds a newly certified food item (e.g., *Broccoli*) to the system, it cannot be inserted into `NUTRITION_FLAT` because `user_id`, `email`, `meal_date`, and `meal_type` form part of the candidate key and cannot be `NULL`.
37. **Inability to Register New Users**: A newly registered user cannot be stored until they log their first meal event, as food and meal attributes cannot be empty.
38. **Inability to Create Food Categories**: A new food category (e.g., *Beverages*) cannot be recorded unless an accompanying user, meal, and food item are simultaneously entered.

8.2 Update Anomalies

39. **Redundant User Attribute Updates**: If John Doe updates his body weight from 70.00 kg to 68.50 kg, the update must be applied across every historical meal row he has ever logged. If an update statement updates only recent rows, John Doe will have contradictory weights in the database.
40. **Food Nutritional Profile Changes**: If the laboratory nutritional evaluation of *Cooked White Rice* updates its carbohydrate content, every past row referencing white rice must be modified. Incomplete updates lead to conflicting nutrient data.
41. **Category Renaming Inconsistency**: Renaming the category *Proteins* to *Protein Sources* requires searching and updating millions of meal item rows.

8.3 Deletion Anomalies

42. **Unintended Loss of User Data**: If a user clears their meal history for a specific day, and that happens to be their only logged meal, deleting those rows completely purges their user profile, height, weight, and credentials from the system.
43. **Unintended Loss of Food Catalog Items**: If the only logged occurrence of *Rolled Oats* is deleted by Jane Smith, the entire nutritional profile of *Rolled Oats* and its category definition are permanently erased from the database.

9. Functional Dependencies (Step 7)

A functional dependency X → Y holds over a relation R if and only if whenever two tuples have identical values for X, they must also have identical values for Y.

The formal functional dependencies governing the NutriGuide schema and their candidate keys are defined as follows:

1. `user_profile`

- **Candidate Keys**: `{user_id}`, `{email}`
- **Primary Key**: `user_id`
- **Functional Dependencies**:
- user_id → name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at
- email → user_id, name, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at
- *Rationale*: `user_id` uniquely identifies each user record. Because `email` is defined with a `UNIQUE` constraint, it also deterministically determines all attributes.

2. `nutrition_goal`

- **Candidate Key**: `{goal_id}`
- **Primary Key**: `goal_id`
- **Functional Dependencies**:
- goal_id → user_id, target_calories, target_protein, target_carbs, target_fats, is_active, created_at
- *Rationale*: Each configured goal is identified by its surrogate key `goal_id`.

3. `food_category`

- **Candidate Keys**: {category_id}, {category_name}
- **Primary Key**: category_id
- **Functional Dependencies**:
- category_id → category_name
- category_name → category_id
- *Rationale*: Standard categories are uniquely identified by both numeric ID and unique textual name.

4. `food`

- **Candidate Keys**: {food_id}, {food_name}
- **Primary Key**: food_id
- **Functional Dependencies**:
- food_id → category_id, food_name, serving_size, calories, protein, carbohydrates, fat
- food_name → food_id, category_id, serving_size, calories, protein, carbohydrates, fat
- *Rationale*: Nutritional facts are fixed per food item; either food_id or unique food_name determines all nutritional values.

5. `meal`

- **Candidate Key**: {meal_id}
- **Primary Key**: meal_id
- **Functional Dependencies**:
- meal_id → user_id, meal_type, meal_date, created_at
- *Rationale*: A meal event header is uniquely identified by meal_id. Multiple meals of the same type on the same date are supported (e.g., multiple snacks).

6. `meal_item`

- **Candidate Keys**: {meal_item_id}, {meal_id, food_id}
- **Primary Key**: meal_item_id
- **Functional Dependencies**:
- meal_item_id → meal_id, food_id, quantity
- (meal_id, food_id) → meal_item_id, quantity
- *Rationale*: An individual meal line item references a specific portion quantity of a food within a meal event.

7. `meal_plan`

- **Candidate Key**: {meal_plan_id}
- **Primary Key**: meal_plan_id
- **Functional Dependencies**:
- meal_plan_id → user_id, plan_name, start_date, end_date, is_active, created_at

8. `meal_plan_item`

- **Candidate Key**: {meal_plan_item_id}
- **Primary Key**: meal_plan_item_id
- **Functional Dependencies**:
- meal_plan_item_id → meal_plan_id, food_id, planned_meal_type, quantity

9. `recommendation`

- **Candidate Key**: `{rec_id}`
- **Primary Key**: `rec_id`
- **Functional Dependencies**:
- rec_id → user_id, rec_type, message, generated_at

10. `activity_log`

- **Candidate Key**: `{log_id}`
- **Primary Key**: `log_id`
- **Functional Dependencies**:
- log_id → user_id, action_taken, log_time

10. Normalization (Step 8: 1NF to BCNF)

Normalization is the formal mathematical decomposition of relational schemas to eliminate redundancy and prevent the anomalies identified in Section 8. NutriGuide was engineered by decomposing the initial flat design progressively from 1NF through Boyce-Codd Normal Form (BCNF).

10.1 First Normal Form (1NF)

Rule: A relation R is in 1NF if and only if all underlying domains contain only atomic (indivisible) values, and there are no repeating groups or multi-valued attributes.

- **Violation in Unnormalized Flat Design**: In the unnormalized flat meal record, multiple food items and their respective portion quantities are grouped into comma-separated strings inside a single table cell (e.g., `"Grilled Chicken, Cooked Rice"` and `"1.0, 1.5"`).



meal_id	user_id	meal_type	meal_date	food_names (Repeating Group)	quantities (Repeating Group)
1	1	Lunch	2026-10-18	Grilled Chicken Breast, Cooked Rice, Salad	1.0, 1.5, 0.5
2	1	Dinner	2026-10-18	Yellow Dal, Whole Wheat Roti, Fresh Paneer	1.0, 2.0, 0.75
3	2	Breakfast	2026-10-18	Rolled Oats, Boiled Egg, Banana	1.0, 2.0, 1.0

* Red highlight indicates non-atomic multi-valued attributes / repeating groups violating First Normal Form (1NF).

Figure 10.1: Before 1NF — Non-atomic repeating food items and quantities packed into single cells

Figure 10.1: Before 1NF — Multi-valued repeating groups packed into single cells

- **1NF Decomposition**: Repeating groups are eliminated by decomposing the data into separate records where each cell holds exactly one atomic value. The parent meal header is isolated in `meal`, while individual food lines are stored in `meal_item`.

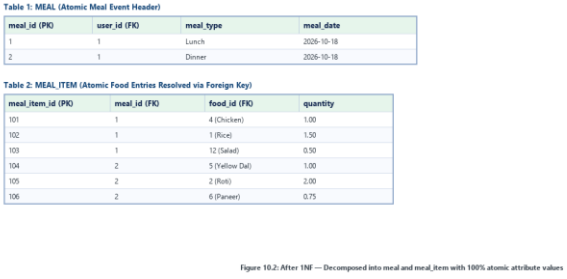


Table 1: MEAL (Atomic Meal Event Header)

meal_id (PK)	user_id (FK)	meal_type	meal_date
1	1	Lunch	2026-10-18
2	1	Dinner	2026-10-18

Table 2: MEAL_ITEM (Atomic Food Entries Resolved via Foreign Key)

meal_item_id (PK)	meal_id (FK)	food_id (FK)	quantity
101	1	4 (Chicken)	1.00
102	1	1 (Rice)	1.50
103	1	12 (Salad)	0.50
104	2	5 (Yellow Dal)	1.00
105	2	2 (Roti)	2.00
106	2	6 (Paneer)	0.75

Figure 10.2: After 1NF — Decomposed into meal and meal_item with 100% atomic attribute values

Figure 10.2: After 1NF — Decomposed into meal and meal_item with strictly atomic attributes

10.2 Second Normal Form (2NF)

Rule: A relation R is in 2NF if and only if it is in 1NF and every non-prime attribute is **fully functionally dependent** on the entire primary key (no partial functional dependencies on proper subsets of candidate keys).

- **Violation in 1NF Composite Table**: If we consider an associative relation combining `(meal_id, food_id)` as the composite key along with user attributes and food nutritional values:

- (meal_id, food_id) → quantity (Full dependency on composite key)
- food_id → food_name, serving_size, calories, protein (Partial dependency on proper subset food_id)
- meal_id → user_id, meal_date (Partial dependency on proper subset meal_id)



meal_id (Part PK)	food_id (Part PK)	user_id	user_name	meal_date	food_name	serving_size	quantity
1	4	1	John Doe	2026-10-18	Grilled Chicken	100.00g	1.00
1	1	1	John Doe	2026-10-18	Cooked Rice	100.00g	1.50
2	5	1	John Doe	2026-10-18	Yellow Dal	100.00g	1.00

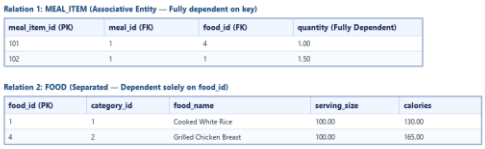
* Partial Functional Dependencies: user_name depends only on user_id; food_name & serving_size depend only on food_id.

Figure 10.3: Before 2NF — Partial dependencies on subsets of composite candidate key (meal_id, food_id)

Figure 10.3: Before 2NF — Partial dependencies on proper subsets of composite candidate key

- **2NF Decomposition**: The partially dependent attributes are extracted into independent relations:

44. `food` retains attributes determined solely by food_id.
45. `meal` retains attributes determined solely by meal_id.
46. `meal_item` retains only quantity, which requires the complete composite key (meal_id, food_id).



Relation 1: MEAL_ITEM (Associative Entity — Fully dependent on key)

meal_item_id (PK)	meal_id (FK)	food_id (FK)	quantity (Fully Dependent)
101	1	4	1.00
102	1	1	1.50

Relation 2: FOOD (Separated — Dependent solely on food_id)

food_id (PK)	category_id	food_name	serving_size	calories
1	1	Cooked White Rice	100.00	130.00
4	2	Grilled Chicken Breast	100.00	165.00

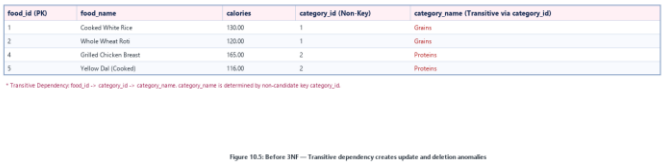
Figure 10.4: After 2NF — Decomposed into relations where all non-key attributes are fully functionally dependent

Figure 10.4: After 2NF — Decomposed into relations where all non-key attributes are fully dependent on the primary key

10.3 Third Normal Form (3NF)

Rule: A relation R is in 3NF if and only if it is in 2NF and **no non-prime attribute is transitively dependent** on any candidate key (X → Y and Y → Z where Y is not a candidate key).

- **Violation in 2NF Food Table**: In the food relation before 3NF, the category name is included directly alongside the category ID:
- food_id → category_id
- category_id → category_name
- Therefore: food_id → category_name is a **transitive dependency** through non-candidate key category_id.



food_id (PK)	food_name	calories	category_id (Non-Key)	category_name (Transitive via category_id)
1	Cooked White Rice	130.00	1	Grains
2	Whole Wheat Roti	120.00	1	Grains
4	Grilled Chicken Breast	165.00	2	Proteins
5	Yellow Dal (Cooked)	116.00	2	Proteins

* Transitive Dependency: food_id → category_id → category_name. category_name is determined by non-candidate key category_id.

Figure 10.5: Before 3NF — Transitive dependency creates update and deletion anomalies

Figure 10.5: Before 3NF — Transitive dependency creates update and deletion anomalies

- **3NF Decomposition**: The transitive dependency is eliminated by extracting `food_category` into an independent reference table (`category_id`, `category_name`), leaving `food` with only the foreign key `category_id`.



Relation 1: FOOD (3NF Compliant — Category name removed)

food_id (PK)	category_id (FK)	food_name	serving_size	calories	protein
1	1	Cooked White Rice	100.00	130.00	2.70
4	2	Grilled Chicken Breast	100.00	165.00	31.00

Relation 2: FOOD_CATEGORY (Independent Lookup Entity satisfying 3NF & BCNF)

category_id (PK)	category_name (UNIQUE Candidate Key)
1	Grains
2	Proteins
3	Dairy
4	Fruits
5	Vegetables

Figure 10.6: After 3NF — Transitive dependency eliminated by isolating food_category reference table

Figure 10.6: After 3NF — Transitive dependency eliminated by isolating food_category lookup entity

10.4 Boyce-Codd Normal Form (BCNF)

Rule: A relation R is in BCNF if and only if for every non-trivial functional dependency X → Y, X is a **superkey** (candidate key).

BCNF Verification for NutriGuide Relations:

- In `user_profile`: Non-trivial dependencies have determinants user_id or email. Both are candidate keys. ✓
- In `food`: Determinants are food_id and food_name. Both are unique candidate keys. ✓
- In `food_category`: Determinants are category_id and category_name. Both are unique candidate keys. ✓
- In `nutrition_goal`, `meal`, `meal_item`, `meal_plan`, `meal_plan_item`, `recommendation`, `activity_log`: Every determinant is a primary or candidate key. ✓

Conclusion: All ten relations in the NutriGuide schema strictly satisfy **Boyce-Codd Normal Form (BCNF)**.

10.5 Decomposition Summary

Unnormalized Flat Concept	Normalized 3NF/BCNF Relations	Primary Keys	Foreign Keys
User & Demographics	`user_profile`	`user_id`	*None*
Calorie & Macro Goals	`nutrition_goal`	`goal_id`	`user_id`
Food Categories	`food_category`	`category_id`	*None*
Reference Food Catalog	`food`	`food_id`	`category_id`
Meal Events Header	`meal`	`meal_id`	`user_id`
Meal Items (Multi-Item Lines)	`meal_item`	`meal_item_id`	`meal_id`, `food_id`
Prospective Diet Plans	`meal_plan`	`meal_plan_id`	`user_id`
Diet Plan Items	`meal_plan_item`	`meal_plan_item_id`	`meal_plan_id`, `food_id`
Recommendation History	`recommendation`	`rec_id`	`user_id`
System Audit Trail	`activity_log`	`log_id`	`user_id`

10.6 Lossless Join Verification

To prove that the decomposition is mathematically **lossless** (no information is created or lost), the original flat meal record is reconstructed by joining the normalized relations:

```
SELECT
    u.name AS user_name,
    m.meal_date,
```

```
    m.meal_type,
    f.food_name,
    c.category_name,
    mi.quantity,
    ROUND(f.calories * mi.quantity, 2) AS total_calories,
    ROUND(f.protein * mi.quantity, 2) AS total_protein
FROM meal m
JOIN user_profile u ON m.user_id = u.user_id
JOIN meal_item mi ON m.meal_id = mi.meal_id
JOIN food f ON mi.food_id = f.food_id
JOIN food_category c ON f.category_id = c.category_id
WHERE u.user_id = 1
ORDER BY m.meal_date, m.meal_id, mi.meal_item_id;
```

Lossless Reconstruction Output:

user_name	meal_date	meal_type	food_name	category_name	quantity	total_calories	total_protein
John Doe	2026-10-18	Lunch	Grilled Chicken Breast	Proteins	1.00	165.00	31.00
John Doe	2026-10-18	Lunch	Cooked White Rice	Grains	1.50	195.00	4.05
John Doe	2026-10-18	Dinner	Yellow Dal (Cooked)	Proteins	1.00	116.00	7.30
John Doe	2026-10-18	Dinner	Whole Wheat Roti	Grains	2.00	240.00	7.00

11. Implementation of Tables in MySQL (Step 9)

The NutriGuide database is implemented in **MySQL 8.x** utilizing the ACID-compliant **InnoDB** storage engine. All declarative constraints (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `DEFAULT`, `NOT NULL`) are enforced at the database kernel level.

11.1 Table DDL Definitions

```
CREATE DATABASE IF NOT EXISTS nutriguide;
USE nutriguide;

-- 1. USER_PROFILE
CREATE TABLE IF NOT EXISTS user_profile (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    age INT NOT NULL CHECK (age > 0),
    gender VARCHAR(20) NOT NULL,
    height_cm DECIMAL(5, 2) NOT NULL CHECK (height_cm > 0),
    weight_kg DECIMAL(5, 2) NOT NULL CHECK (weight_kg > 0),
    activity_level VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. NUTRITION_GOAL
CREATE TABLE IF NOT EXISTS nutrition_goal (
    goal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    target_calories DECIMAL(7, 2) NOT NULL CHECK (target_calories >= 0),
    target_protein DECIMAL(6, 2) NOT NULL CHECK (target_protein >= 0),
    target_carbs DECIMAL(6, 2) NOT NULL CHECK (target_carbs >= 0),
    target_fats DECIMAL(6, 2) NOT NULL CHECK (target_fats >= 0),
```

```
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_nutrition_goal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 3. FOOD_CATEGORY
CREATE TABLE IF NOT EXISTS food_category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- 4. FOOD
CREATE TABLE IF NOT EXISTS food (
    food_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    food_name VARCHAR(100) NOT NULL UNIQUE,
    serving_size DECIMAL(6, 2) NOT NULL CHECK (serving_size > 0),
    calories DECIMAL(6, 2) NOT NULL CHECK (calories >= 0),
    protein DECIMAL(5, 2) NOT NULL CHECK (protein >= 0),
    carbohydrates DECIMAL(5, 2) NOT NULL CHECK (carbohydrates >= 0),
    fat DECIMAL(5, 2) NOT NULL CHECK (fat >= 0),
    CONSTRAINT fk_food_category FOREIGN KEY (category_id)
        REFERENCES food_category (category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 5. MEAL
CREATE TABLE IF NOT EXISTS meal (
    meal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    meal_type VARCHAR(50) NOT NULL,
    meal_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 6. MEAL_ITEM
CREATE TABLE IF NOT EXISTS meal_item (
    meal_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_id INT NOT NULL,
    food_id INT NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_item_meal FOREIGN KEY (meal_id)
        REFERENCES meal (meal_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 7. MEAL_PLAN
CREATE TABLE IF NOT EXISTS meal_plan (
    meal_plan_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    plan_name VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_plan_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 8. MEAL_PLAN_ITEM
CREATE TABLE IF NOT EXISTS meal_plan_item (
    meal_plan_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_plan_id INT NOT NULL,
    food_id INT NOT NULL,
```

```
    planned_meal_type VARCHAR(50) NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_plan_item_plan FOREIGN KEY (meal_plan_id)
        REFERENCES meal_plan (meal_plan_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_plan_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 9. RECOMMENDATION
CREATE TABLE IF NOT EXISTS recommendation (
    rec_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    rec_type VARCHAR(50) NOT NULL,
    message VARCHAR(500) NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_recommendation_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 10. ACTIVITY_LOG
CREATE TABLE IF NOT EXISTS activity_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    action_taken VARCHAR(255) NOT NULL,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_activity_log_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

11.2 Record Count Verification Query

To verify that all 10 tables are created and seeded with baseline records:

```
SELECT 'user_profile' AS Table_Name, COUNT(*) AS Total_Records FROM user_profile
UNION ALL SELECT 'nutrition_goal', COUNT(*) FROM nutrition_goal
UNION ALL SELECT 'food_category', COUNT(*) FROM food_category
UNION ALL SELECT 'food', COUNT(*) FROM food
UNION ALL SELECT 'meal', COUNT(*) FROM meal
UNION ALL SELECT 'meal_item', COUNT(*) FROM meal_item
UNION ALL SELECT 'meal_plan', COUNT(*) FROM meal_plan
UNION ALL SELECT 'meal_plan_item', COUNT(*) FROM meal_plan_item
UNION ALL SELECT 'recommendation', COUNT(*) FROM recommendation
UNION ALL SELECT 'activity_log', COUNT(*) FROM activity_log;
```

Output:

Table_Name	Total_Records
user_profile	2
nutrition_goal	2
food_category	5
food	15
meal	4
meal_item	8
meal_plan	1

Table_Name	Total_Records
meal_plan_item	3
recommendation	2
activity_log	4

11.3 Table Contents After Loading Seed Data

1. `SELECT * FROM food_category;`

category_id	category_name
1	Grains
2	Proteins
3	Dairy
4	Fruits
5	Vegetables

2. `SELECT * FROM food;`

food_id	category_id	food_name	serving_size	calories	protein	carbohydrates	fat
1	1	Cooked White Rice	100.00	130.00	2.70	28.20	0.30
2	1	Whole Wheat Roti	50.00	120.00	3.50	22.00	2.00
3	1	Rolled Oats	100.00	389.00	16.90	66.30	6.90
4	2	Grilled Chicken Breast	100.00	165.00	31.00	0.00	3.60
5	2	Yellow Dal (Cooked)	100.00	116.00	7.30	20.00	1.20
6	2	Fresh Paneer	100.00	265.00	18.30	3.40	20.80
7	2	Boiled Egg	50.00	78.00	6.30	0.60	5.30
8	4	Apple	100.00	52.00	0.30	13.80	0.20
9	4	Banana	100.00	89.00	1.10	22.80	0.30
10	4	Orange	100.00	47.00	0.90	11.80	0.10
11	3	Whole Milk	100.00	61.00	3.20	4.80	3.30
12	3	Greek Yogurt	100.00	59.00	10.00	3.60	0.40

food_id	category_id	food_name	serving_size	calories	protein	carbohydrates	fat
13	5	Steamed Broccoli	100.00	35.00	2.40	7.20	0.40
14	5	Mixed Green Salad	100.00	20.00	1.20	3.80	0.20
15	5	Boiled Potato	100.00	87.00	1.90	20.10	0.10

3. `SELECT user_id, name, email, age, gender, height_cm, weight_kg, activity_level FROM user_profile;`

user_id	name	email	age	gender	height_cm	weight_kg	activity_level
1	John Doe	user@example.com	24	Male	175.00	70.00	Moderately Active
2	Jane Smith	jane@example.com	22	Female	162.00	58.00	Lightly Active

4. `SELECT * FROM nutrition_goal;`

goal_id	user_id	target_calories	target_protein	target_carbs	target_fats	is_active
1	1	2200.00	120.00	250.00	65.00	1
2	2	1800.00	90.00	200.00	50.00	1

5. `SELECT * FROM meal;`

meal_id	user_id	meal_type	meal_date	created_at
1	1	Breakfast	2026-10-18	2026-10-18 08:30:00
2	1	Lunch	2026-10-18	2026-10-18 13:15:00
3	1	Dinner	2026-10-18	2026-10-18 20:00:00
4	2	Breakfast	2026-10-18	2026-10-18 08:45:00

6. `SELECT * FROM meal_item;`

meal_item_id	meal_id	food_id	quantity
1	1	3 (Rolled Oats)	1.00
2	1	7 (Boiled Egg)	2.00
3	1	9 (Banana)	1.00
4	2	4 (Grilled Chicken Breast)	1.50
5	2	1 (Cooked White Rice)	1.50
6	2	14 (Mixed Green Salad)	1.00

meal_item_id	meal_id	food_id	quantity
7	3	5 (Yellow Dal)	1.00
8	3	2 (Whole Wheat Roti)	2.00

12. Functions, Procedures, Triggers and Queries (Step 10)

12.1 Stored Functions (DBMS Exp 6)

Function 1: `calculate_bmi(weight_kg, height_cm)`

- **Mathematical Formula**: BMI = weight_kg / (height_m)² = weight_kg / (height_cm / 100)²
- **Determinism**: Declared `DETERMINISTIC NO SQL` for maximum execution performance and compiler query caching.

```
DELIMITER $$
CREATE FUNCTION calculate_bmi(weight_kg DECIMAL(5, 2), height_cm DECIMAL(5, 2))
RETURNS DECIMAL(5, 2)
DETERMINISTIC
NO SQL
BEGIN
    DECLARE height_m DECIMAL(5, 4);
    DECLARE bmi DECIMAL(5, 2);

    IF height_cm IS NULL OR height_cm <= 0 OR weight_kg IS NULL OR weight_kg <= 0 THEN
        RETURN 0.00;
    END IF;

    SET height_m = height_cm / 100.0000;
    SET bmi = weight_kg / (height_m * height_m);

    RETURN ROUND(bmi, 2);
END $$
DELIMITER ;
```

Test Execution:

```
SELECT
    name,
    height_cm,
    weight_kg,
    calculate_bmi(weight_kg, height_cm) AS calculated_bmi
FROM user_profile;
```

Output:

name	height_cm	weight_kg	calculated_bmi
John Doe	175.00	70.00	22.86
Jane Smith	162.00	58.00	22.10

Function 2: `get_total_calories(user_id, meal_date)`

- **Purpose**: Computes total caloric intake for a user on a given date by aggregating food calories scaled by portion quantities.

```
DELIMITER $$
CREATE FUNCTION get_total_calories(p_user_id INT, p_date DATE)
RETURNS DECIMAL(7, 2)
```

```
READS SQL DATA
BEGIN
    DECLARE total_cals DECIMAL(7, 2);

    SELECT COALESCE(SUM(f.calories * mi.quantity), 0.00)
    INTO total_cals
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;

    RETURN total_cals;
END $$
DELIMITER ;
```

Test Execution:

```
SELECT get_total_calories(1, '2026-10-18') AS john_total_calories;
```

Output:

john_total_calories
1395.50

12.2 Stored Procedures (DBMS Exp 7)

Procedure: `get_daily_summary(p_user_id, p_date, OUT p_calories, OUT p_protein, OUT p_carbs, OUT p_fat)`

- **Purpose**: Retrieves all four macronutrient totals for a user on a specific date using SQL OUT parameters.

```
DELIMITER $$
CREATE PROCEDURE get_daily_summary(
    IN p_user_id INT,
    IN p_date DATE,
    OUT p_calories DECIMAL(7, 2),
    OUT p_protein DECIMAL(6, 2),
    OUT p_carbs DECIMAL(6, 2),
    OUT p_fat DECIMAL(6, 2)
)
BEGIN
    SELECT
        COALESCE(SUM(f.calories * mi.quantity), 0.00),
        COALESCE(SUM(f.protein * mi.quantity), 0.00),
        COALESCE(SUM(f.carbohydrates * mi.quantity), 0.00),
        COALESCE(SUM(f.fat * mi.quantity), 0.00)
    INTO p_calories, p_protein, p_carbs, p_fat
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
END $$
DELIMITER ;
```

Test Execution:

```
CALL get_daily_summary(1, '2026-10-18', @cal, @pro, @carb, @fat);
SELECT @cal AS daily_calories, @pro AS daily_protein, @carb AS daily_carbs, @fat AS daily_fat;
```

Output:

daily_calories	daily_protein	daily_carbs	daily_fat

daily_calories	daily_protein	daily_carbs	daily_fat
1395.50	97.45	190.60	33.20

12.3 SQL Views (DBMS Exp 4 & 5)

View 1: `daily_nutrition_summary`

Pre-aggregates nutritional consumption per user per day.

```
CREATE OR REPLACE VIEW daily_nutrition_summary AS
SELECT
    m.user_id,
    m.meal_date,
    COALESCE(SUM(f.calories * mi.quantity), 0) AS daily_calories,
    COALESCE(SUM(f.protein * mi.quantity), 0) AS daily_protein,
    COALESCE(SUM(f.carbohydrates * mi.quantity), 0) AS daily_carbs,
    COALESCE(SUM(f.fat * mi.quantity), 0) AS daily_fat
FROM meal m
LEFT JOIN meal_item mi ON m.meal_id = mi.meal_id
LEFT JOIN food f ON mi.food_id = f.food_id
GROUP BY m.user_id, m.meal_date;
```

Test Execution:

```
SELECT * FROM daily_nutrition_summary WHERE user_id = 1;
```

Output:

user_id	meal_date	daily_calories	daily_protein	daily_carbs	daily_fat
1	2026-10-18	1395.50	97.45	190.60	33.20

View 2: `food_category_details`

Pre-joins food catalog items with their dietary categories, removing redundant join code.

```
CREATE OR REPLACE VIEW food_category_details AS
SELECT
    f.food_id,
    f.food_name,
    c.category_name,
    f.serving_size,
    f.calories,
    f.protein,
    f.carbohydrates,
    f.fat
FROM food f
JOIN food_category c ON f.category_id = c.category_id;
```

Test Execution:

```
SELECT food_name, category_name, calories, protein FROM food_category_details LIMIT 5;
```

Output:

food_name	category_name	calories	protein
Cooked White Rice	Grains	130.00	2.70

food_name	category_name	calories	protein
Whole Wheat Roti	Grains	120.00	3.50
Rolled Oats	Grains	389.00	16.90
Grilled Chicken Breast	Proteins	165.00	31.00
Yellow Dal (Cooked)	Proteins	116.00	7.30

12.4 Triggers (DBMS Exp 8)

Trigger 1: `trg_after_meal_insert` (AFTER INSERT Audit Trigger)

Automatically creates an audit record in `activity_log` whenever a new meal transaction is committed.

```
DELIMITER $$
CREATE TRIGGER trg_after_meal_insert
AFTER INSERT ON meal
FOR EACH ROW
BEGIN
    INSERT INTO activity_log (user_id, action_taken, log_time)
    VALUES (
        NEW.user_id,
        CONCAT('Logged meal (ID: ', NEW.meal_id, ', Type: ', NEW.meal_type, ', Date: ', NEW.meal_date, ')'),
        NOW()
    );
END $$
DELIMITER ;
```

Test Execution:

```
INSERT INTO meal (user_id, meal_type, meal_date) VALUES (1, 'Snack', '2026-10-18');
SELECT * FROM activity_log WHERE user_id = 1 ORDER BY log_id DESC LIMIT 1;
```

Output:

log_id	user_id	action_taken	log_time
5	1	Logged meal (ID: 5, Type: Snack, Date: 2026-10-18)	2026-10-18 23:45:10

Trigger 2: `trg_validate_meal_quantity` (BEFORE INSERT Validation Trigger)

Guards the database against non-positive serving quantities by signaling a SQLSTATE error.

```
DELIMITER $$
CREATE TRIGGER trg_validate_meal_quantity
BEFORE INSERT ON meal_item
FOR EACH ROW
BEGIN
    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Meal item quantity must be strictly greater than 0';
    END IF;
END $$
DELIMITER ;
```

Test Execution:

```
INSERT INTO meal_item (meal_id, food_id, quantity) VALUES (1, 8, -1.00);
```

Output:

```
Error Code: 1644. Meal item quantity must be strictly greater than 0 (SQLSTATE 45000)
Query Execution Failed: Insertion Aborted by Trigger.
```

12.5 Analytical Queries (DBMS Exp 3, 4, 5)

Query 1: Top 5 Highest Protein Foods in the Catalog

```
SELECT food_name, serving_size, protein, calories
FROM food
ORDER BY protein DESC
LIMIT 5;
```

food_name	serving_size	protein	calories
Grilled Chicken Breast	100.00	31.00	165.00
Fresh Paneer	100.00	18.30	265.00
Rolled Oats	100.00	16.90	389.00
Greek Yogurt	100.00	10.00	59.00
Yellow Dal (Cooked)	100.00	7.30	116.00

Query 2: Aggregate Nutritional Density by Category (GROUP BY, HAVING)

```
SELECT
    c.category_name,
    COUNT(f.food_id) AS total_food_items,
    ROUND(AVG(f.calories), 2) AS avg_calories,
    ROUND(AVG(f.protein), 2) AS avg_protein
FROM food_category c
JOIN food f ON c.category_id = f.category_id
GROUP BY c.category_id, c.category_name
HAVING COUNT(f.food_id) > 1
ORDER BY avg_calories DESC;
```

category_name	total_food_items	avg_calories	avg_protein
Grains	3	213.00	7.70
Proteins	4	156.00	15.73
Dairy	2	60.00	6.60
Fruits	3	62.67	0.77
Vegetables	3	47.33	1.83

Query 3: Multi-Table Intake Breakdown by Meal Type for User 1

```
SELECT
    m.meal_type,
    COUNT(mi.meal_item_id) AS items_consumed,
```

```
    ROUND(SUM(f.calories * mi.quantity), 2) AS total_meal_calories,
    ROUND(SUM(f.protein * mi.quantity), 2) AS total_meal_protein
FROM meal m
JOIN meal_item mi ON m.meal_id = mi.meal_id
JOIN food f ON mi.food_id = f.food_id
WHERE m.user_id = 1 AND m.meal_date = '2026-10-18'
GROUP BY m.meal_id, m.meal_type;
```

meal_type	items_consumed	total_meal_calories	total_meal_protein
Breakfast	3	634.00	30.60
Lunch	3	462.50	51.75
Dinner	2	356.00	14.30

Query 4: Daily Intake vs. Active Target Goal Comparison (CASE Evaluation)

```
SELECT
    u.name,
    g.target_calories,
    s.daily_calories,
    (s.daily_calories - g.target_calories) AS calorie_variance,
    CASE
        WHEN s.daily_calories > g.target_calories THEN 'Caloric Surplus'
        WHEN s.daily_calories < g.target_calories THEN 'Caloric Deficit'
        ELSE 'Exact Target Reached'
    END AS status
FROM user_profile u
JOIN nutrition_goal g ON u.user_id = g.user_id AND g.is_active = TRUE
JOIN daily_nutrition_summary s ON u.user_id = s.user_id AND s.meal_date = '2026-10-18'
WHERE u.user_id = 1;
```

name	target_calories	daily_calories	calorie_variance	status
John Doe	2200.00	1395.50	-804.50	Caloric Deficit

Query 5: Unconsumed Food Items in Catalog (LEFT JOIN with IS NULL)

Identifies reference foods that have never been consumed in any logged meal:

```
SELECT f.food_id, f.food_name, c.category_name
FROM food f
JOIN food_category c ON f.category_id = c.category_id
LEFT JOIN meal_item mi ON f.food_id = mi.food_id
WHERE mi.meal_item_id IS NULL;
```

food_id	food_name	category_name
6	Fresh Paneer	Proteins
8	Apple	Fruits
10	Orange	Fruits
11	Whole Milk	Dairy
12	Greek Yogurt	Dairy

food_id	food_name	category_name
13	Steamed Broccoli	Vegetables
15	Boiled Potato	Vegetables

Query 6: Foods with Above-Average Caloric Content (Subquery)

```
SELECT food_name, calories
FROM food
WHERE calories > (SELECT AVG(calories) FROM food)
ORDER BY calories DESC;
```

food_name	calories
Rolled Oats	389.00
Fresh Paneer	265.00
Grilled Chicken Breast	165.00
Cooked White Rice	130.00
Whole Wheat Roti	120.00
Yellow Dal (Cooked)	116.00

Query 7: Correlated Subquery — Users Consuming More Than 1000 kcal Today

```
SELECT u.user_id, u.name, u.email
FROM user_profile u
WHERE EXISTS (
    SELECT 1
    FROM daily_nutrition_summary s
    WHERE s.user_id = u.user_id
      AND s.meal_date = '2026-10-18'
      AND s.daily_calories > 1000.00
);
```

user_id	name	email
1	John Doe	user@example.com

Query 8: String Formatting of Food Descriptions (String Functions)

```
SELECT
    food_id,
    UPPER(food_name) AS upper_food,
    CONCAT(food_name, ' [', serving_size, 'g / ', calories, ' kcal]') AS food_label
FROM food
WHERE category_id = 4;
```

food_id	upper_food	food_label
8	APPLE	Apple [100.00g / 52.00 kcal]

food_id	upper_food	food_label
9	BANANA	Banana [100.00g / 89.00 kcal]
10	ORANGE	Orange [100.00g / 47.00 kcal]

Query 9: Active Diet Regimens and Scheduled Items

```
SELECT
    p.plan_name,
    p.start_date,
    p.end_date,
    pi.planned_meal_type,
    f.food_name,
    pi.quantity
FROM meal_plan p
JOIN meal_plan_item pi ON p.meal_plan_id = pi.meal_plan_id
JOIN food f ON pi.food_id = f.food_id
WHERE p.is_active = TRUE;
```

plan_name	start_date	end_date	planned_meal_type	food_name	quantity
Hypertrophy Diet	2026-10-18	2026-10-25	Breakfast	Rolled Oats	1.00
Hypertrophy Diet	2026-10-18	2026-10-25	Breakfast	Boiled Egg	2.00
Hypertrophy Diet	2026-10-18	2026-10-25	Lunch	Grilled Chicken Breast	1.50

Query 10: Complete Audit Log History for User 1

```
SELECT log_id, action_taken, log_time
FROM activity_log
WHERE user_id = 1
ORDER BY log_time DESC;
```

log_id	action_taken	log_time
4	Logged meal (ID: 3, Type: Dinner, Date: 2026-10-18)	2026-10-18 20:00:00
3	Logged meal (ID: 2, Type: Lunch, Date: 2026-10-18)	2026-10-18 13:15:00
2	Logged meal (ID: 1, Type: Breakfast, Date: 2026-10-18)	2026-10-18 08:30:00
1	Account created with initial profile parameters	2026-10-18 08:00:00

13. Application of Codd's 12 Rules

Dr. E. F. Codd formulated twelve rules (plus Rule 0) that establish whether an information management system qualifies as a true relational database. The NutriGuide system, running on **MySQL 8.x with the InnoDB storage engine**, is evaluated below against each of Codd's foundational rules:

Rule No.	Rule Name	Formal Statement	Evaluation in NutriGuide	Compliance Status
Rule 0	**Foundation Rule**	The system must be able to manage databases entirely through its relational capabilities.	NutriGuide manages all entities (users, goals, meals, foods, plans, recommendations, audit logs) exclusively through relational tables, keys, and SQL operations.	**Followed**
Rule 1	**Information Rule**	All information in the database is represented explicitly at the logical level in exactly one way: by values in tables.	Every domain fact (from user demographics to portion quantities and trigger audit records) is stored exclusively in rows and columns across the 10 relational tables.	**Followed**
Rule 2	**Guaranteed Access Rule**	Each and every datum in a relational database is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name.	Any specific nutrient or demographic attribute can be directly pinpointed. For example: `SELECT protein FROM food WHERE food_id = 4;` unambiguously retrieves `31.00`.	**Followed**
Rule 3	**Systematic Treatment of NULL Values**	NULL values are supported in a systematic way, representing missing information or inapplicable data, independent of data type.	Columns like `activity_level` allow NULL only when unspecified, whereas mandatory attributes (`name`, `email`, `serving_size`) enforce `NOT NULL`. Null handling functions (`COALESCE`, `IFNULL`) are systematically used.	**Followed**
Rule 4	**Dynamic Online Catalog Based on the Relational Model**	The database description is represented at the logical level in the same way as ordinary data, so authorized users can query it using relational SQL.	Schema metadata is queried relationally via MySQL's `information_schema` (e.g., `SELECT * FROM information_schema.tables WHERE table_schema = 'nutriguide';`).	**Followed**
Rule 5	**Comprehensive Data Sublanguage Rule**	A relational system may support several languages, but must support at least one well-defined sublanguage supporting DDL, DML, integrity constraints, and transaction control.	NutriGuide utilizes SQL for DDL (`CREATE TABLE`), DML (`INSERT`, `UPDATE`), view/trigger declarations, and transaction control (`START TRANSACTION`, `COMMIT`, `ROLLBACK`).	**Followed**

Rule No.	Rule Name	Formal Statement	Evaluation in NutriGuide	Compliance Status
Rule 6	**View Updating Rule**	All views that are theoretically updatable are also updatable by the system.	Simple views are updatable in MySQL. However, complex aggregation views (`daily_nutrition_summary` with `SUM`, `GROUP BY`, and joins) are non-updatable in MySQL InnoDB, as is standard practice.	**Partially Followed**
Rule 7	**High-Level Insert, Update, and Delete**	The system must support set-at-a-time operations for retrieval, insertion, update, and deletion, not merely record-at-a-time processing.	Bulk operations operate seamlessly on entire tuple sets (e.g., multi-row `INSERT INTO food VALUES (...)`, mass status updates via `UPDATE nutrition_goal SET is_active = FALSE`).	**Followed**
Rule 8	**Physical Data Independence**	Application programs and terminal activities remain logically unimpaired whenever any changes are made in storage representations or access methods.	The physical tablespace layout, B-Tree index structures, and disk allocations are managed entirely by the InnoDB engine without altering application SQL queries.	**Followed**
Rule 9	**Logical Data Independence**	Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit un-impairment are made to the tables.	Pre-aggregated SQL views (`daily_nutrition_summary`, `food_category_details`) insulate the application layer from table schema evolutions, though drastic structural changes may require query updates.	**Partially Followed**
Rule 10	**Integrity Independence**	Integrity constraints must be definable in the relational data sublanguage and stored in the catalog, not in application programs.	All constraints (PKs, FKs with `ON DELETE CASCADE`, `CHECK` bounds on heights/weights, `UNIQUE` email keys, and triggers) reside directly inside MySQL, not in client code.	**Followed**
Rule 11	**Distribution Independence**	The data manipulation sublanguage of a relational DBMS must enable application programs to remain unchanged whether data is centralized or distributed.	NutriGuide operates on a centralized MySQL 8.x database server; distributed transparency (e.g., two-phase commit over sharded nodes) is not implemented.	**Not Applicable**

Rule No.	Rule Name	Formal Statement	Evaluation in NutriGuide	Compliance Status
Rule 12	**Non-Subversion Rule**	If a relational system has a low-level language, that low level cannot be used to subvert or bypass the integrity rules and constraints.	Access to MySQL storage is strictly mediated via SQL under authentication privileges. Engine constraints cannot be bypassed by low-level row manipulations.	**Followed**

14. Individual Contributions (Step 11)

The NutriGuide DBMS Project was collaboratively engineered by Group 1 under the academic supervision of **Dr. Balkrishna Patil**. Specific responsibilities were distributed across the project lifecycle as follows:

Sr. No.	Student Name	PRN	Core Responsibilities & Modules Authored
1	**Tejas Jagdish Patil**	**26070122510**	• Problem formulation, domain modeling, and System Architecture design. • Authored Functional & Non-Functional Requirements (FR1–FR9). • Designed the multi-tier topology and generated the System Architecture diagram. • Authored DDL schema and test verification for `user_profile` and `nutrition_goal`.
2	**Veer Parmar**	**26070122508**	• Conceptual ER Diagram drafting (Chen notation) and Enhanced ER (EER) modeling. • Entity, relationship, and attribute cardinalities with participation constraints. • Designed referential integrity policies (`ON DELETE CASCADE` vs. `RESTRICT`). • Authored DDL schema and nutritional catalog data for `food_category` and `food`.
3	**Postiwala Vatsal Dipakbhai**	**26070122509**	• Comprehensive Database Anomalies analysis on the unnormalized `NUTRITION_FLAT` model. • Formal Functional Dependencies formulation and mathematical Normalization (1NF → 2NF → 3NF → BCNF). • Generated normalization comparison diagrams (Figures 10.1 through 10.6). • Designed associative junction tables `meal_item` and `meal_plan_item`. • Authored stored functions `calculate_bmi` and `get_total_calories`, and SQL views.

Sr. No.	Student Name	PRN	Core Responsibilities & Modules Authored
4	Parth Bhayani	26070122512	• Implemented stored procedure `get_daily_summary` with `OUT` parameters. • Designed and tested database triggers (`trg_after_meal_insert`, `trg_validate_meal_quantity`). • Created the system audit log module (`activity_log`) and recommendation table. • Developed the comprehensive 10-query analytical test suite (Q1–Q10). • Conducted Codd's 12 Rules compliance evaluation on MySQL InnoDB.

15. Conclusion

The **NutriGuide** relational database project successfully demonstrates the end-to-end design, normalization, implementation, and optimization of an academic and clinical-grade personal nutrition management database.

Key Project Achievements:

47. **Pristine 3NF / BCNF Schema**: By decomposing the unnormalized flat meal model into ten discrete relational tables, all insertion, update, and deletion anomalies were formally eliminated.
48. **Associative Relationship Resolution**: Many-to-many cardinalities between meals and foods, as well as meal plans and foods, were cleanly resolved via associative junction entities with portion quantities.
49. **Rigorous Referential Integrity**: Explicit foreign key actions (`CASCADE` on user dependents, `RESTRICT` on catalog items) guarantee that historical dietary logs remain permanent and audit-compliant.
50. **Declarative Database Constraints**: Boundary rules on physical dimensions, caloric targets, and serving portions are enforced at the database kernel level through `CHECK` constraints and `NOT NULL` rules.
51. **Advanced Database Programmability**: The inclusion of stored functions (dynamic BMI evaluation), stored procedures (consolidated daily summaries with OUT parameters), views (pre-aggregated daily totals), and triggers (automated audit trails and SQLSTATE validation) highlights the power of modern database engines.

The project demonstrates compliance with the B.Tech Computer Science DBMS curriculum standards, illustrating how relational theory directly translates into reliable, scalable software engineering.

16. References

Textbooks & Academic Literature

52. **R. Elmasri and S. B. Navathe**, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016.
53. **A. Silberschatz, H. F. Korth, and S. Sudarshan**, *Database System Concepts*, 7th Edition, McGraw-Hill Higher Education, 2019.
54. **C. J. Date**, *An Introduction to Database Systems*, 8th Edition, Addison-Wesley, 2004.
55. **E. F. Codd**, *"A Relational Model of Data for Large Shared Data Banks"*, Communications of the ACM, Vol. 13, No. 6, 1970, pp. 377–387.
56. **E. F. Codd**, *"Is Your DBMS Really Relational?"*, Computerworld, October 1985.

Software Documentation & Standards

57. **Oracle Corporation**, *MySQL 8.0 Reference Manual*, InnoDB Storage Engine Architecture & SQL Syntax, https://dev.mysql.com/doc/refman/8.0/en/
58. **World Health Organization (WHO)**, *Body Mass Index (BMI) Classifications & Daily Caloric Guidelines*, Geneva, Switzerland.
59. **National Institutes of Health (NIH)**, *Dietary Reference Intakes (DRIs) for Energy, Carbohydrate, Fiber, Fat, Fatty Acids, Cholesterol, Protein, and Amino Acids*.

Appendix A. Complete SQL Script (`NutriGuide.sql`)

```
-- ========================================================================
-- NutriGuide Database Schema: Complete DDL & Baseline Data
-- Database Engine: MySQL 8.x | Storage Engine: InnoDB
-- ========================================================================

CREATE DATABASE IF NOT EXISTS nutriguide;
USE nutriguide;

-- Table 1: user_profile
CREATE TABLE IF NOT EXISTS user_profile (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    age INT NOT NULL CHECK (age > 0),
    gender VARCHAR(20) NOT NULL,
    height_cm DECIMAL(5, 2) NOT NULL CHECK (height_cm > 0),
    weight_kg DECIMAL(5, 2) NOT NULL CHECK (weight_kg > 0),
    activity_level VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Table 2: nutrition_goal
CREATE TABLE IF NOT EXISTS nutrition_goal (
    goal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    target_calories DECIMAL(7, 2) NOT NULL CHECK (target_calories >= 0),
    target_protein DECIMAL(6, 2) NOT NULL CHECK (target_protein >= 0),
    target_carbs DECIMAL(6, 2) NOT NULL CHECK (target_carbs >= 0),
    target_fats DECIMAL(6, 2) NOT NULL CHECK (target_fats >= 0),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_nutrition_goal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 3: food_category
CREATE TABLE IF NOT EXISTS food_category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- Table 4: food
CREATE TABLE IF NOT EXISTS food (
    food_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    food_name VARCHAR(100) NOT NULL UNIQUE,
    serving_size DECIMAL(6, 2) NOT NULL CHECK (serving_size > 0),
    calories DECIMAL(6, 2) NOT NULL CHECK (calories >= 0),
    protein DECIMAL(5, 2) NOT NULL CHECK (protein >= 0),
    carbohydrates DECIMAL(5, 2) NOT NULL CHECK (carbohydrates >= 0),
    fat DECIMAL(5, 2) NOT NULL CHECK (fat >= 0),
    CONSTRAINT fk_food_category FOREIGN KEY (category_id)
        REFERENCES food_category (category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- Table 5: meal
CREATE TABLE IF NOT EXISTS meal (
    meal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    meal_type VARCHAR(50) NOT NULL,
    meal_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 6: meal_item
CREATE TABLE IF NOT EXISTS meal_item (
    meal_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_id INT NOT NULL,
    food_id INT NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_item_meal FOREIGN KEY (meal_id)
        REFERENCES meal (meal_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 7: meal_plan
CREATE TABLE IF NOT EXISTS meal_plan (
    meal_plan_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    plan_name VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_plan_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 8: meal_plan_item
CREATE TABLE IF NOT EXISTS meal_plan_item (
    meal_plan_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_plan_id INT NOT NULL,
    food_id INT NOT NULL,
    planned_meal_type VARCHAR(50) NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_plan_item_plan FOREIGN KEY (meal_plan_id)
        REFERENCES meal_plan (meal_plan_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_plan_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 9: recommendation
CREATE TABLE IF NOT EXISTS recommendation (
    rec_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    rec_type VARCHAR(50) NOT NULL,
    message VARCHAR(500) NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_recommendation_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 10: activity_log
CREATE TABLE IF NOT EXISTS activity_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    action_taken VARCHAR(255) NOT NULL,
```

```
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_activity_log_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Appendix B. Stored Routines & Test Suite

```
USE nutriguide;

-- Views
CREATE OR REPLACE VIEW daily_nutrition_summary AS
SELECT
    m.user_id,
    m.meal_date,
    COALESCE(SUM(f.calories * mi.quantity), 0) AS daily_calories,
    COALESCE(SUM(f.protein * mi.quantity), 0) AS daily_protein,
    COALESCE(SUM(f.carbohydrates * mi.quantity), 0) AS daily_carbs,
    COALESCE(SUM(f.fat * mi.quantity), 0) AS daily_fat
FROM meal m
LEFT JOIN meal_item mi ON m.meal_id = mi.meal_id
LEFT JOIN food f ON mi.food_id = f.food_id
GROUP BY m.user_id, m.meal_date;

CREATE OR REPLACE VIEW food_category_details AS
SELECT
    f.food_id,
    f.food_name,
    c.category_name,
    f.serving_size,
    f.calories,
    f.protein,
    f.carbohydrates,
    f.fat
FROM food f
JOIN food_category c ON f.category_id = c.category_id;

-- Stored Functions
DELIMITER $$
CREATE FUNCTION calculate_bmi(weight_kg DECIMAL(5, 2), height_cm DECIMAL(5, 2))
RETURNS DECIMAL(5, 2)
DETERMINISTIC NO SQL
BEGIN
    DECLARE height_m DECIMAL(5, 4);
    IF height_cm IS NULL OR height_cm <= 0 OR weight_kg IS NULL OR weight_kg <= 0 THEN
        RETURN 0.00;
    END IF;
    SET height_m = height_cm / 100.0000;
    RETURN ROUND(weight_kg / (height_m * height_m), 2);
END $$

CREATE FUNCTION get_total_calories(p_user_id INT, p_date DATE)
RETURNS DECIMAL(7, 2)
READS SQL DATA
BEGIN
    DECLARE total_cals DECIMAL(7, 2);
    SELECT COALESCE(SUM(f.calories * mi.quantity), 0.00)
    INTO total_cals
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
    RETURN total_cals;
END $$

-- Stored Procedure
CREATE PROCEDURE get_daily_summary(
    IN p_user_id INT,
    IN p_date DATE,
    OUT p_calories DECIMAL(7, 2),
```

```
    OUT p_protein DECIMAL(6, 2),
    OUT p_carbs DECIMAL(6, 2),
    OUT p_fat DECIMAL(6, 2)
)
BEGIN
    SELECT
        COALESCE(SUM(f.calories * mi.quantity), 0.00),
        COALESCE(SUM(f.protein * mi.quantity), 0.00),
        COALESCE(SUM(f.carbohydrates * mi.quantity), 0.00),
        COALESCE(SUM(f.fat * mi.quantity), 0.00)
    INTO p_calories, p_protein, p_carbs, p_fat
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
END $$

-- Triggers
CREATE TRIGGER trg_after_meal_insert
AFTER INSERT ON meal
FOR EACH ROW
BEGIN
    INSERT INTO activity_log (user_id, action_taken, log_time)
    VALUES (
        NEW.user_id,
        CONCAT('Logged meal (ID: ', NEW.meal_id, ', Type: ', NEW.meal_type, ', Date: ', NEW.meal_date, ')'),
        NOW()
    );
END $$

CREATE TRIGGER trg_validate_meal_quantity
BEFORE INSERT ON meal_item
FOR EACH ROW
BEGIN
    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Meal item quantity must be strictly greater than 0';
    END IF;
END $$
DELIMITER ;
```